

Mouth Mechanics - Fuse, Fail-Closed, Clearance-Only Act

How Gate talks to Velaru and why /v1/act never executes the irreversible write.

Source: `gate/app.py` (excerpts)

What to understand

- `velaru_fuse` - HTTP proxy to the Velaru fuse engine (hop/lookup)
- `fail_closed` - timeouts and bad responses become DENY/halt, never soft LIVE
- `run_welded_act` - clearance permit only; `write_executed` is always false from Gate

Law

Clearance is not execution. The exclusive edge (worker / weld) is the only place the write may run after permit.

fail_closed

Uncertainty and transport failure must not become a silent yes.

```
def fail_closed(reason: str, fuse_id=None):
    """OCSP-shaped: unreachable is HALT, never LIVE. Relying party must not act."""
    payload = {
        "ok": False,
        "halt": True,
        "fail_closed": True,
        "ocsp": True,
        "verdict": False,
        "state": "UNREACHABLE",
        "acted": False,
        "message": "Relying party MUST halt. Timeout or upstream failure is not LIVE.",
        "reason": reason,
        "fuse_id": fuse_id,
        "charge": "CHARGE webhook is the only DEAD->LIVE path on the engine.",
        "spec": "gate-ocsp-fuse-v1",
    }
    bound.attach(payload, 503)
    headers = {
        "Cache-Control": "no-store",
        "X-Gate-Fail-Closed": "1",
        "X-Gate-Halt": "1",
    }
    return payload, 503, headers
```

velaru_fuse

All fuse questions go through a fail-closed client. Non-JSON / 5xx / timeout => halt.

```
def velaru_fuse(method: str, path: str, fuse_id=None, **kwargs):
    try:
        r = velaru_request(method, path, timeout=OCSP_TIMEOUT, **kwargs)
    except requests.Timeout:
        return fail_closed("ocsp_timeout", fuse_id)
```

```
except requests.RequestException as e:
    return fail_closed(f"upstream_unreachable:{e}", fuse_id)
if r.status_code >= 500:
    return fail_closed(f"upstream_{r.status_code}", fuse_id)
try:
    data = r.json()
except ValueError:
    return fail_closed("upstream_non_json", fuse_id)
return data, r.status_code, {}
```

run_welded_act

Returns whether the act may proceed. Gate does not move money, bind, or release here. acted=true means may proceed; write_executed stays false.

```
def run_welded_act(fuse_id: str, action: str):
    """Clearance permit only. Gate never executes the irreversible write here.

    `acted` means clearance would allow the exclusive door to proceed - not that
    money left or a bind committed. `write_executed` is always false from Gate.
    """
    hop, status, extra = velaru_fuse("POST", "/api/v1/fuse/hop", fuse_id=fuse_id, json={"fuse_id":
fuse_id})
    extra = dict(extra or {})
    extra["X-Gate-Closed-World"] = "1"
    extra["X-Gate-Write-Executed"] = "0"

    if status >= 500 or (isinstance(hop, dict) and hop.get("halt")):
        if isinstance(hop, dict):
            hop["acted"] = False
            hop["write_executed"] = False
            hop["side_effect"] = False
            hop["clearance_only"] = True
            hop["action"] = action
            hop["welded"] = True
            hop["closed_world"] = True
            bound.attach(hop, status, closed_world=True)
        return hop, status, extra

    allowed = bool(isinstance(hop, dict) and hop.get("verdict") is True)
    result = {
        "spec": "gate-welded-act-v2",
        "welded": True,
        "closed_world": True,
        "clearance_only": True,
        "side_effect": False,
        "write_executed": False,
        "action": action,
        "acted": allowed,
        "clearance_allows": allowed,
        "halt": not allowed,
        "fuse_id": fuse_id,
        "hop": hop,
        "message": (
            "Clearance permit - hop LIVE/verdict true. Gate did not execute the write. "
            "Exclusive door (worker/Gosu) must enforce; bypass is out of protocol."
        )
    }
```

```
        if allowed
        else "Clearance refuse - DEAD or verdict false. No side door. write_executed=false."
    ),
    "their_production": False,
}
extra["X-Gate-Acted"] = "1" if allowed else "0"
bound.attach(result, 200, closed_world=True)
return result, 200, extra

@app.route("/demo/act", methods=["POST"])
```